


# Accelerating and Scaling Python for HPC Science Integration

Dr Vassil Vassilev, CERN

compiler-research.org

# YOUR PYTHON — CPython interprets and orchestrates

ecosystem: NumPy · pandas · ML stack · your analysis code

## AOT · vertical

**Packaged libraries**

NumPy · CuPy · mpi4py · Dragon

```
cp.add(x, y)
```

**glue:** shipped
**kernel:** shipped
**when:** before install

## AOT · vertical

**Bindings you write**

ctypes · pybind11 · Cython

```
mylib.fast_add(x, y)
```

**glue:** you write it
**kernel:** existing C/C++
**when:** your build step

## JIT · vertical

**Compiled Python**

Numba — @njit · @cuda.jit · JAX - jax.jit

```
add(x, y) # @njit
```

**glue:** generated
**kernel:** Python → LLVM
**when:** JIT, first call

## JIT · horizontal

**4 · Runtime interop**

**cppjit (CppInterOp)**

```
cppjit.gbl.add(x, y)
```

**glue:** built from C++ AST
**kernel:** C++ cppdef'd live
**when:** JIT, first use

*no shaft of its own — objects, templates, inheritance, callbacks cross both ways*

## C++ ECOSYSTEM

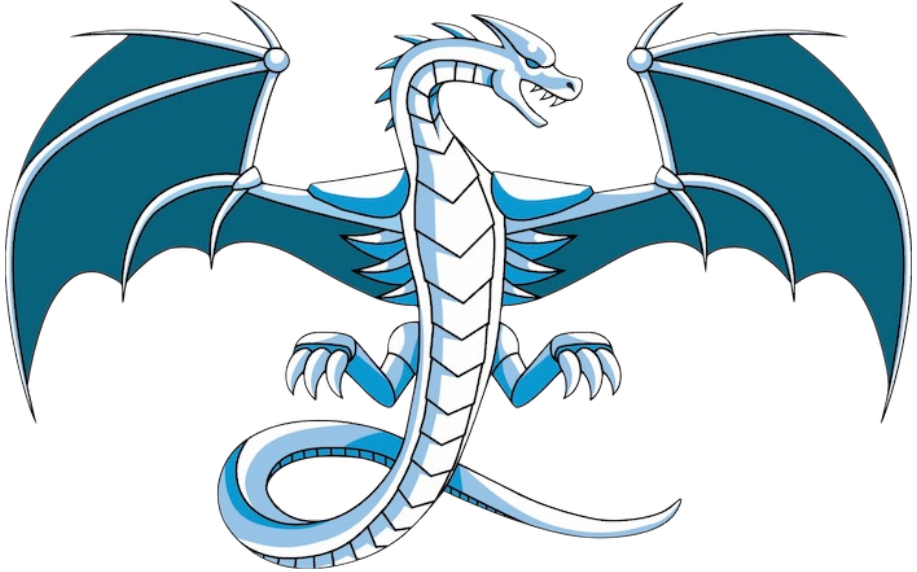
Eigen · CUDA
30 years of HPC code
your lab's libraries

compiles to native like everything else

# COMPILED NATIVE CODE — kernels run with the GIL released

CPU SIMD · GPU (CUDA) · interconnect (MPI) · cluster processes

# The compiler is in the club



```
import cppjit
cppjit.include("kernel/logistic_step.h")   # the header, as written

N = 50.0
for _ in range(60):
    N = cppjit.gbl.logistic_step["double"](N, 0.30, 2000.0)
print(N)                                   # 1999.999842435456
```

A single session where compilers, languages and tools compose: call, transform, compose.

# Call it: not just fast but zero-copy

```
view = np.asarray(vec.data())

view.flags["OWNDATA"]                              # False
hex(view.ctypes.data) == hex(cppjit.addressof(vec.data()))   # True

view[0] = -1.0;  assert vec[0]  == -1.0      # write visible C++-side
vec[0]  = 50.0;  assert view[0] ==  50.0     # write visible NumPy-side
```

# Transform it: Until model has no .backward()

```
def step_jax(N, r, K): return N + r * N * (1 - N / K)# the law, in JAX
jax.grad(step_jax, argnums=1)(50.0, 0.3, 2000.0)      # 48.75


cppjit.include("kernel/logistic_step.h")              # the law, as written
jax.grad(lambda r: cppjit.gbl.logistic_step["double"](50.0, r,
2000.0))(0.3)
# TypeError: Could not find "logistic_step<double>"
```

Deep learning gave Python the derivative of anything you write *in the framework*. Many big sciences frameworks (ROOT, Geant4, BioDynaMo) are not written in the framework, and the tracer cannot enter a compiled function.

# Transform it: **C++ source**, dumped verbatim

```
double step_darg1(double N, double r, double K) {
          double _d_N = 0; double _d_r = 1; double _d_K = 0;
          clad::ValueAndPushforward<double, double> _t0 =
              logistic_step_pushforward(N, r, K, _d_N, _d_r, _d_K);
          return _t0.pushforward;
      }
```

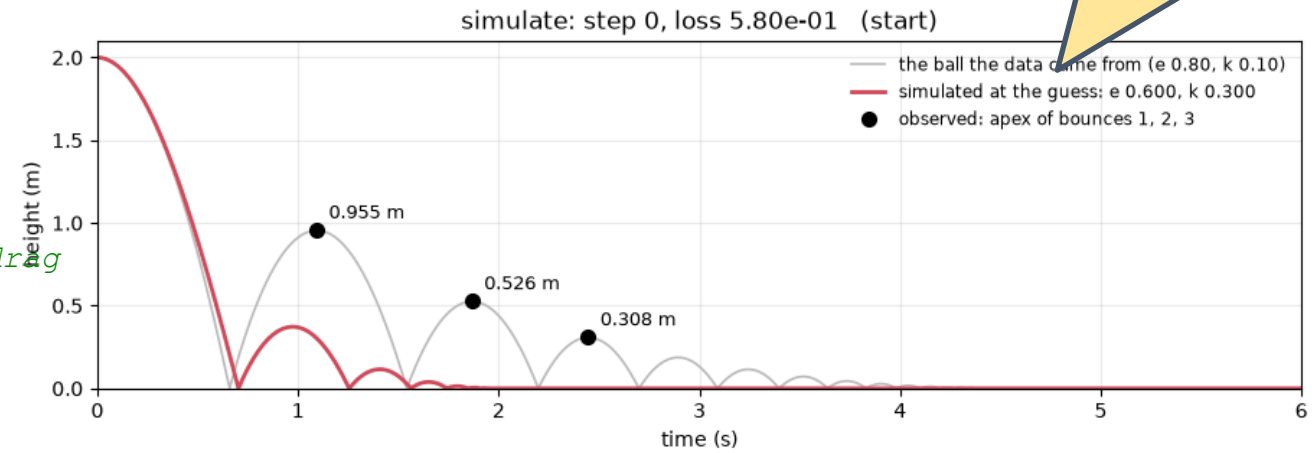
```
# Clad rides inside the session's compiler
os.environ["CPPINTEROP_EXTRA_INTERPRETER_ARGS"] = "-fplugin=clad.so -O2"
import cppjit; cppjit.cppdef(cell) # the cell above, compiled in the session
```

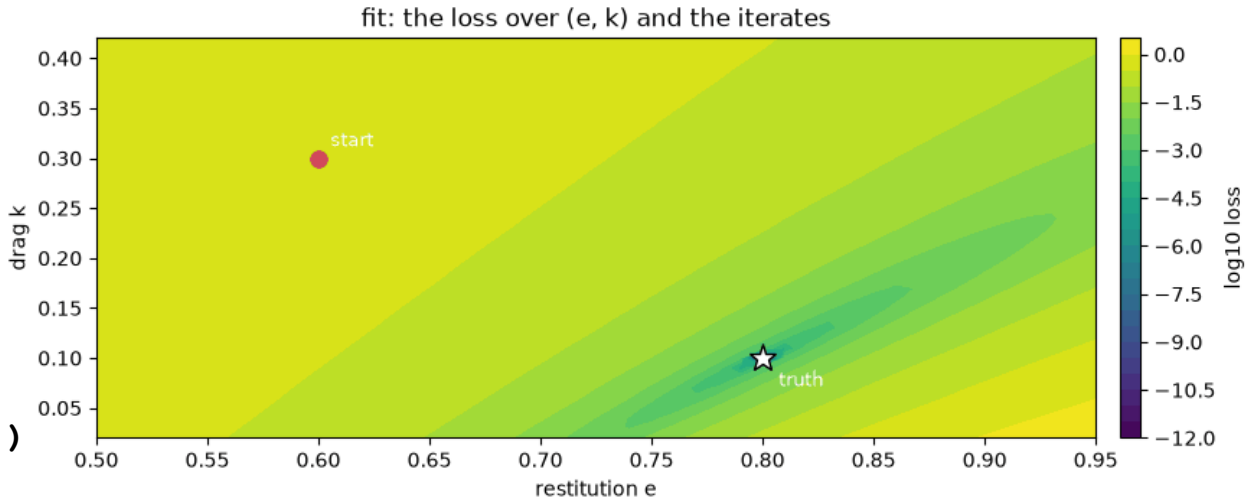
# Simulate and fit it

**BFGS in JAX with Clad Gradient**

```
double apex(double e, double k, int n) {
  // drop from 2 m; apex of bounce n
  double y = 2.0, v = 0.0, dt = 1e-3, top = 0.0;
  int bounces = 0;
  for (int i = 0; i < 20000; ++i) {
    v -= (9.81 + k * v * std::fabs(v)) * dt;// gravity, air drag
    y += v * dt;
    if (y < 0) { y = 0; v = -e * v; ++bounces; }
    if (bounces == n && v > 0 && y > top) top = y;
  }
  return top;
}
// p = {e, k}; obs = the three apex heights
double loss(double* p, const double* obs) {
  double L = 0; for (int n = 1; n <= 3; ++n) {
  double d = apex(p[0], p[1], n) - obs[n-1]; L += d*d; }
  return L; }
auto d_loss = clad::gradient(loss, "p");

# forward: loss(); backward: d_loss
cpp_loss = jax.custom_vjp(...)
# JAX runs the optimiser; it never sees the C++
fit = optx.minimise(cpp_loss, optx.BFGS(), [0.6, 0.3])
```




Observed: the first three bounces reach 0.955, 0.526 and 0.308 m; unknown: the restitution e and the drag k. The cell is compiled by cppjit in the session, with Clad loaded as a compiler plugin.

# Autodiff with Clad: .backward() for ”any” code

```
// clang -fplugin=/.../libclad.so
// Necessary for clad to work include
#include "clad/Differentiator/Differentiator.h"
double pow2(double x) { return x * x; }

double pow2_darg0(double); // to be filled by clad

int main() {
 auto dfdx = clad::differentiate(pow2, 0);
 // Function execution can happen in 3 ways:
 // 1) Using CladFunction::execute method.
 double res = dfdx.execute(3);

 printf("res is: %f\n", res);
 printf("The derivative code is: %s\n", dfdx.getCode());
 return res;
}
```

**Clang Plugin**

**Tells the plugin to create pow2_darg0.**

COMPILER EXPLORER

https://godbolt.org/z/h54nxjWc6

```
performance-test@vv-nuc ~/clad-build-llvm11 $ ./a.out
6.000000
The derivative code is: double pow2_darg0(double x) {
    double _d_x = 1;
    return _d_x * x + x * _d_x;
}
```

# Case Studies

**The ball setup at scale. A** model you can run, a few measurements, unknown dials, a loss, and the compiler's derivative of that loss through the model:

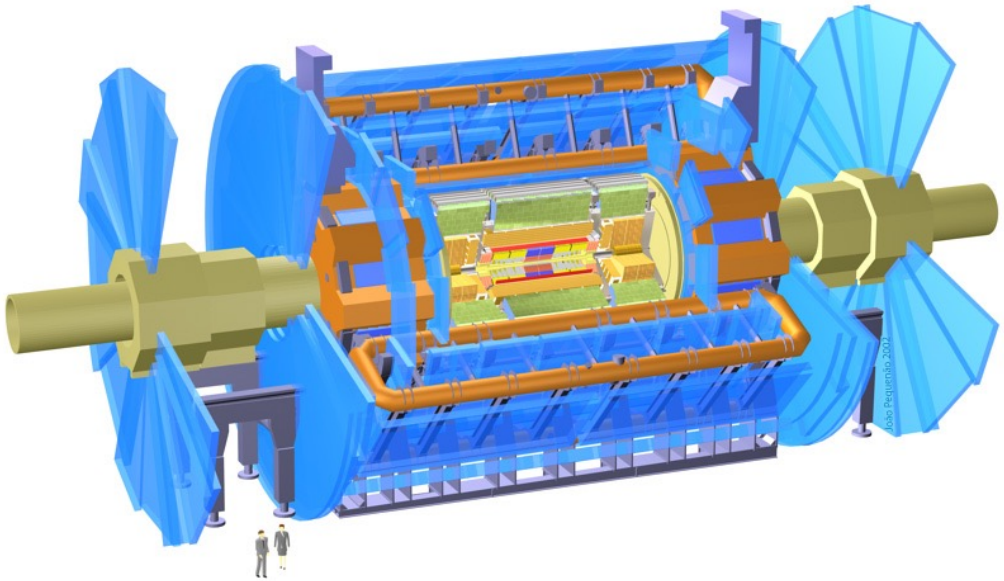
- Data Bound: High Energy Physics
  *C++ likelihoods over enormous datasets; fits with hundreds of parameters*
- Compute Bound: Cancer Modeling
  *A mechanistic agent-based model of CAR T-cell therapy, C++ on BioDynaMo*

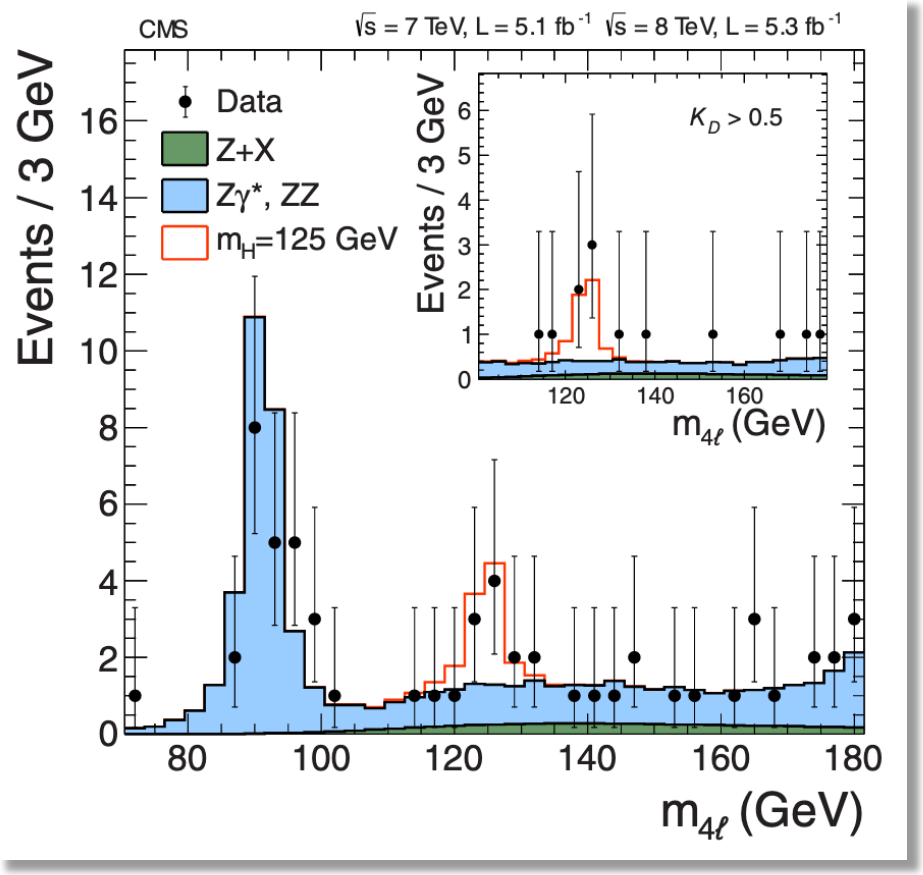


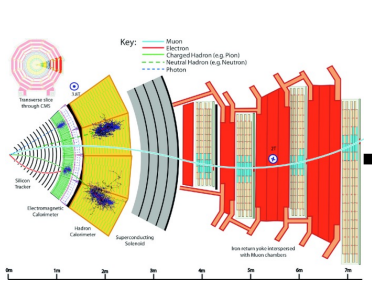
# Example 1: High-energy physics

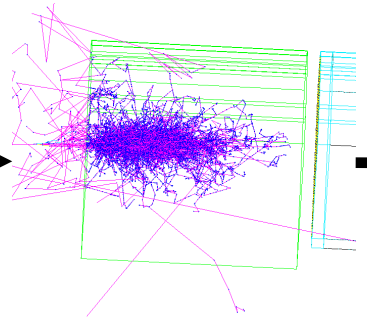
The pressure is data. Call the likelihood as written; transform it with Clad; compose it with JAX.

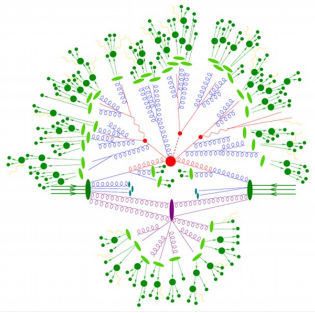
# Particles Whisper

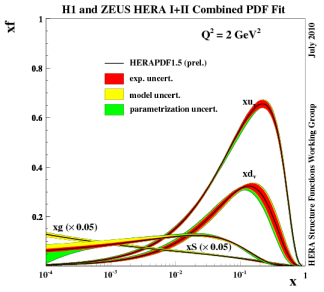


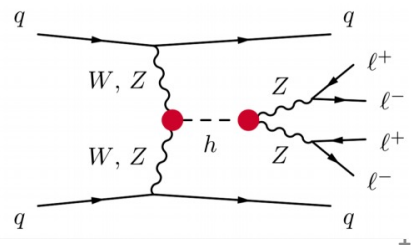








# The ball equation but

## Likelihoods are central for High Energy Physics

**Same thing as the ball, at scale:** the likelihood is the loss, the physics parameters are the dials, the recorded events are the data.

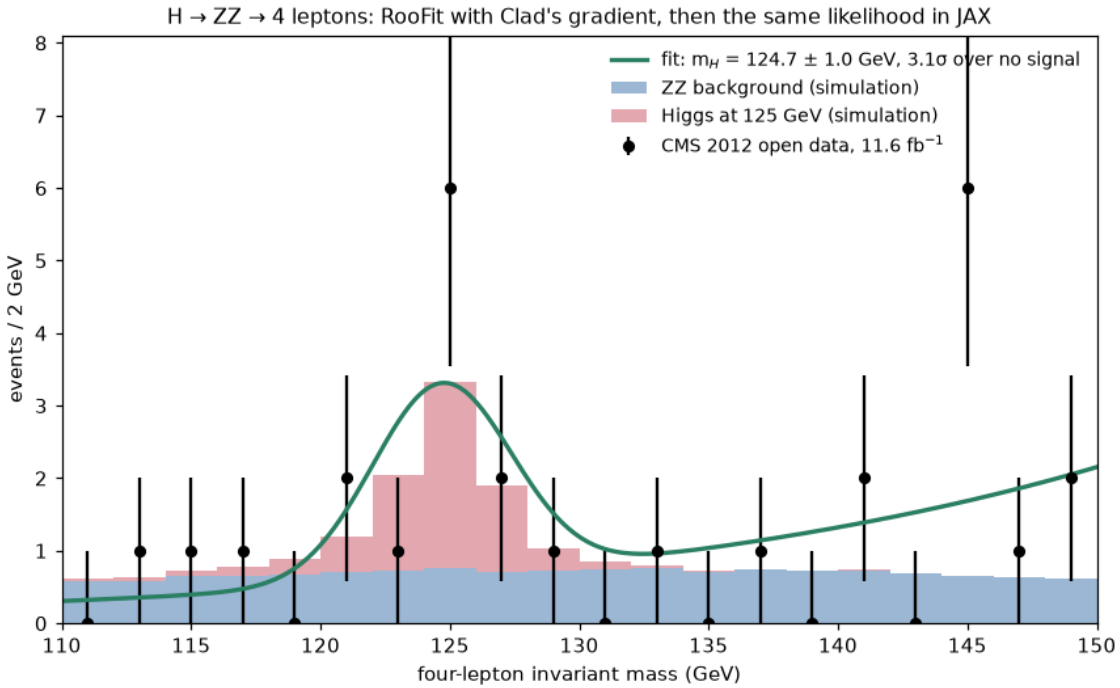
$$L(\vec{n}, \vec{a}|\vec{\eta}, \vec{\chi}) = \prod_{c \in unbinned\ ch} \prod_{i \in obs} \frac{f_c(\vec{x}_{ci}|\vec{\eta}, \vec{\chi})}{\int f_c(\vec{x}_{ci}|\vec{\eta}, \vec{\chi})\, d\vec{x}_c} \cdot \prod_{c \in binned\ ch(analytical)} \prod_{b \in obs} Pois(n_{cb}|\nu(\vec{\eta}, \vec{\chi})) \cdot \prod_{\chi \in \vec{\chi}} c_\chi(a_\chi|\chi)$$

$$\vec{n} : data, \vec{a} : auxilary\ data, \vec{\eta} : unconstrained\ parameters, \vec{\chi} : constrained\ parameters$$

CMS Combine Paper https://arxiv.org/pdf/2404.06614

# Discover the Higgs boson, yourself

```
nll = model.createNLL(data, EvalBackend.Codegen())   # the likelihood out as C++; Clad writes its gradient
RooMinimizer(nll).minimize("Minuit2", "migrad")      # the standard minimiser, with that gradient
roofit_nll = jax.custom_vjp(...)                     # JAX: value and Clad's gradient, through a callback
optx.minimise(roofit_nll, optx.BFGS(), start)        # JAX's optimiser; it never sees RooFit
```

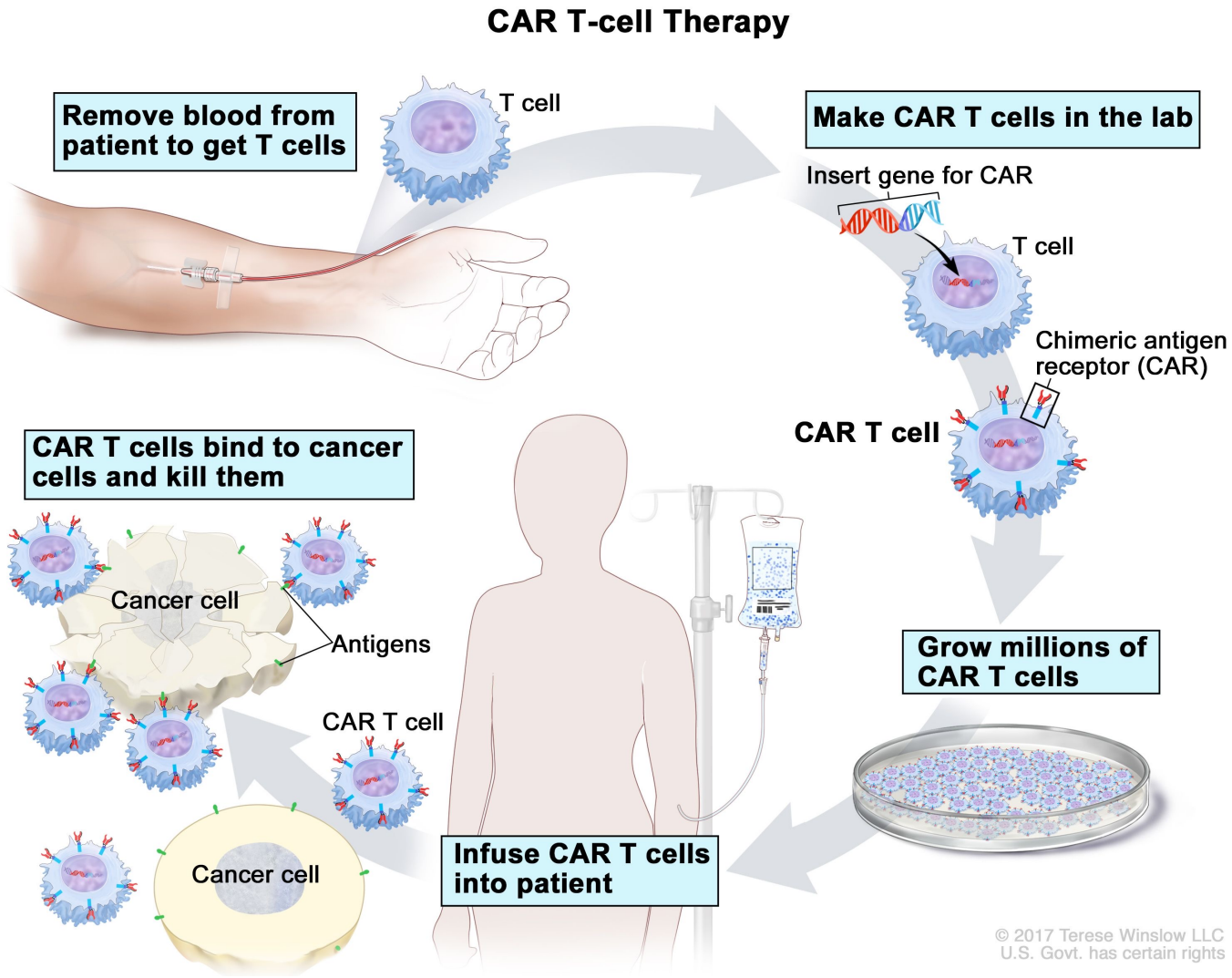



CMS 2012 open data, 11.6 $fb^{-1}$, the published four-lepton selection (ROOT's own tutorial of the CMS outreach analysis): 28 candidates between 110 and 150 GeV.
RooFit emits the likelihood as C++, Clad writes its gradient, Minuit2 finds $m_h$ = 124.7 ± 1.0 GeV, 9 ± 4 signal events, 3.1σ over no signal, in 62 ms.
Then JAX: the same likelihood as a function with a derivative, its own optimiser, the same minimum to four decimals in 57 steps. Nothing was ported.
Local significance, one window, one selection; the 2012 observation combined more channels and data.

# Example 2: Cancer research

The pressure is computation. Call the model as built; compose it from Python; transform its oxygen update, and fit it.

# Cell Engineering




Effective in **leukemia** and other **blood** cancers. However, CAR T still **remains limited in solid tumors** due to unique tumor microenvironmental factors.

CARTopiaX

# The Problem

- How a tumour organoid responds to CAR T-cell therapy: engineered immune cells that find, bind and kill cancer cells
- A mechanistic agent-based model: every cancer cell and every CAR T-cell is an agent in a 3-D tumour, with its own growth, death, movement and the oxygen it gets (Luque et al., Sci. Rep. 2024)
- **CARTopiaX**: the model implemented in C++17 on BioDynaMo, plus a calibration loop, exploration tools and a Python interface; the model supplies the rules, the platform the infrastructure
- 3,957 cells at the start, 12,372 after 10 days, each updated every 0.1 min with two $50^3$ oxygen and signal grids: 14,400 steps a day, 432,000 for the published 30-day protocol; 0.9 h per simulated day here
- From python: the same 109 ms a step as the model's own binary (113 ms); 6 s of start-up once

# Same Story

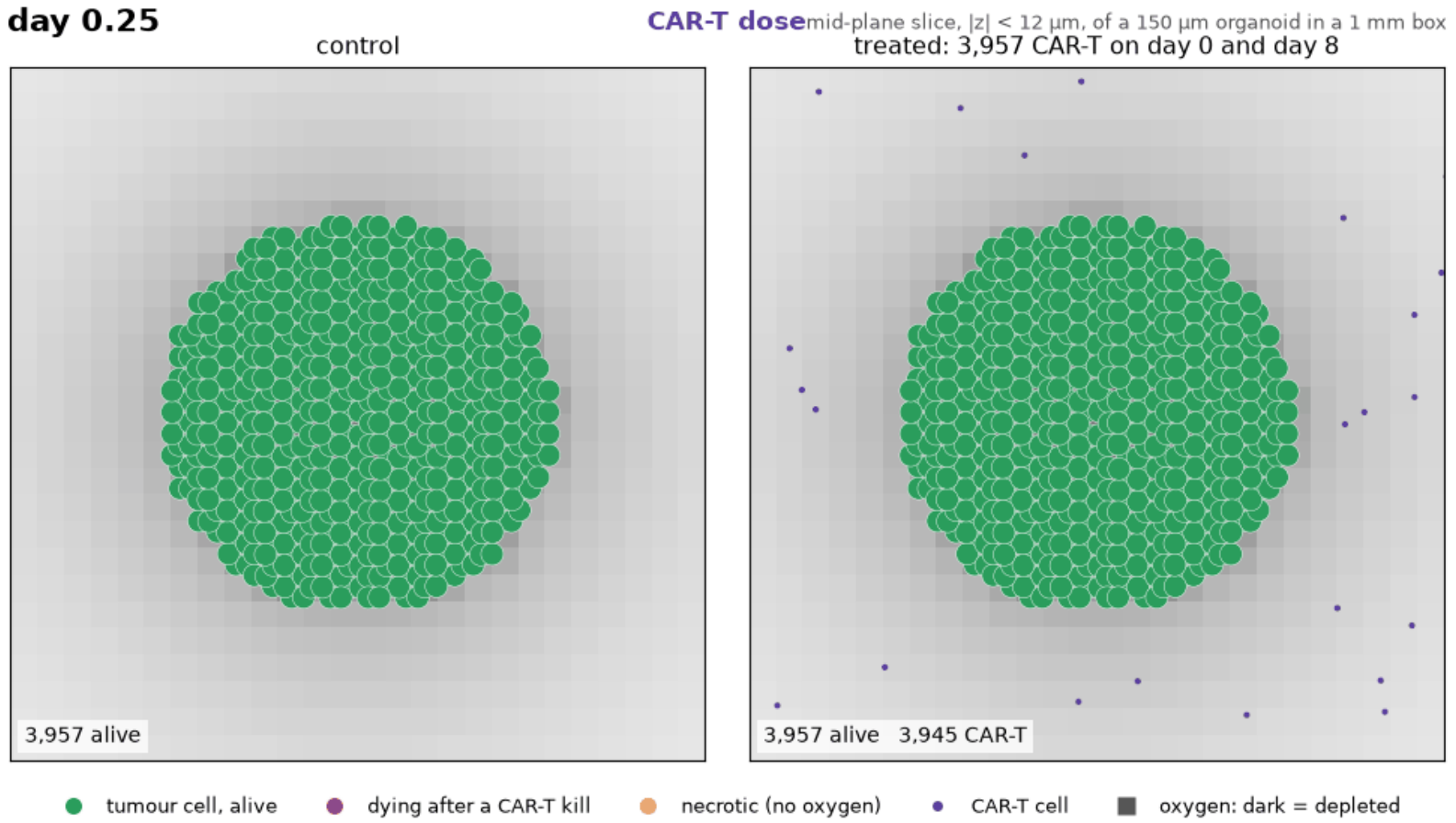
```
import cppjit, ctypes
ctypes.CDLL("libbiodynamo.so", ctypes.RTLD_GLOBAL)    # the platform
ctypes.CDLL("libCARTopiaX.so", ctypes.RTLD_GLOBAL)    # the model, as built
cppjit.add_include_path("CARTopiaX/src")
cppjit.cppdef(open("organoid.cpp").read()) # its headers + build, dose, step, count

run = cppjit.gbl.sc26.run_organoid          # (minutes, every, radius_um, dose, seed)
control = run(14400, 360, 150, 0,    42)
treated = run(14400, 360, 150, 3957, 42)
# after 240 h: 12,372 alive tumour cells untreated, 721 treated
```

The model itself, from the session, through CARTopiaX: 12,372 living tumour cells in the untreated organoid after 10 simulated days, 721 in the one dosed with 3,957 CAR T-cells (114 killed, 94% fewer alive). Headers parsed in 6 s; the run averaged 0.23 s a step on 4 threads as the organoid grew, and one simulated day is 14,400 steps — this is why it is C++.

# Modelling CAR T-Cell Therapy

CARTopiaX



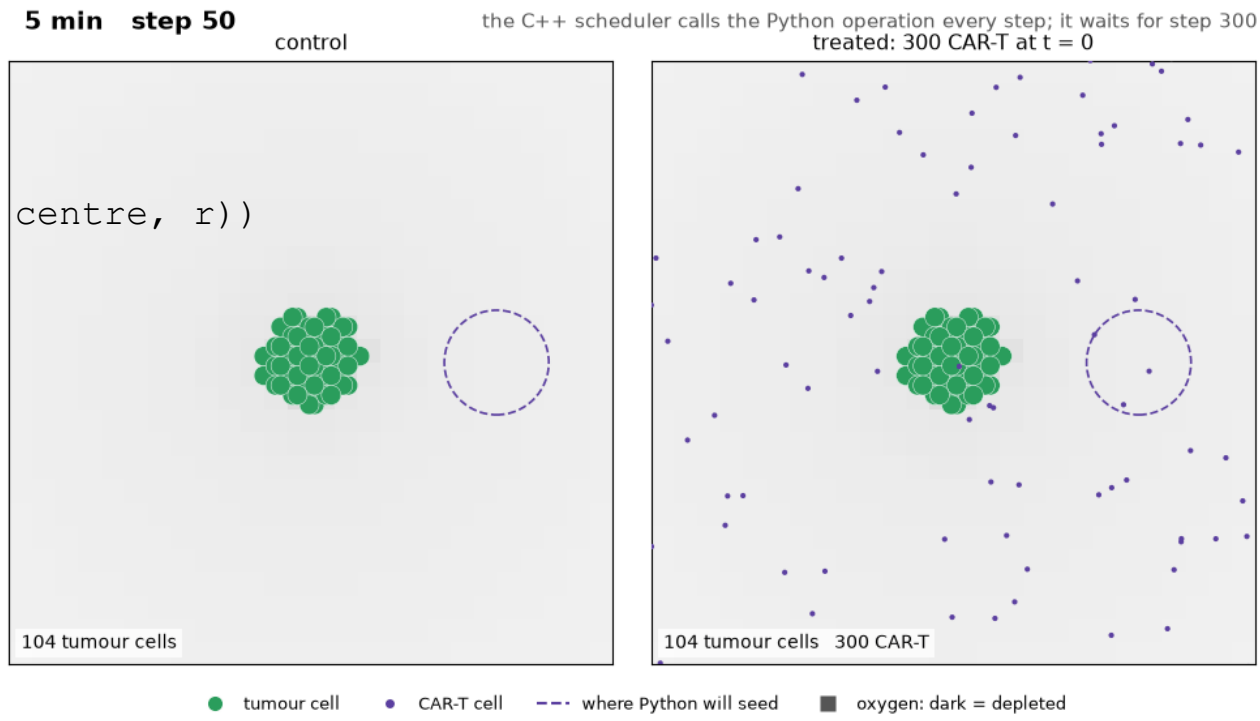

*Luque's CAR T-cell model (Luque et al., Sci. Rep. 2024), run through CARTopiaX, the BioDynaMo platform built around it. The published protocol: a 150 µm organoid, 3,957 CAR T-cells at day 0 and day 8; after 10.0 simulated days, 12,372 living tumour cells untreated against 721 treated. The curve you saw fitted two slides ago came out of this same model at small size (60 µm), from this session. Every dot is one cell with its own rules: a tumour cell reads the oxygen in its voxel and divides, rests or dies by it; a CAR T-cell moves up a signal the tumour secretes, binds a neighbour within 14 µm and kills with a probability; the oxygen field is solved every substep. Nothing here is a curve fitted to data: it is the mechanism, run. Day 10 is the trough, not the outcome: by then the CAR T-cells are nearly spent, and in the model (and in CARTopiaX's own 30-day replication of the paper) the tumour regrows once they are gone.*

# A second organoid from a Python class

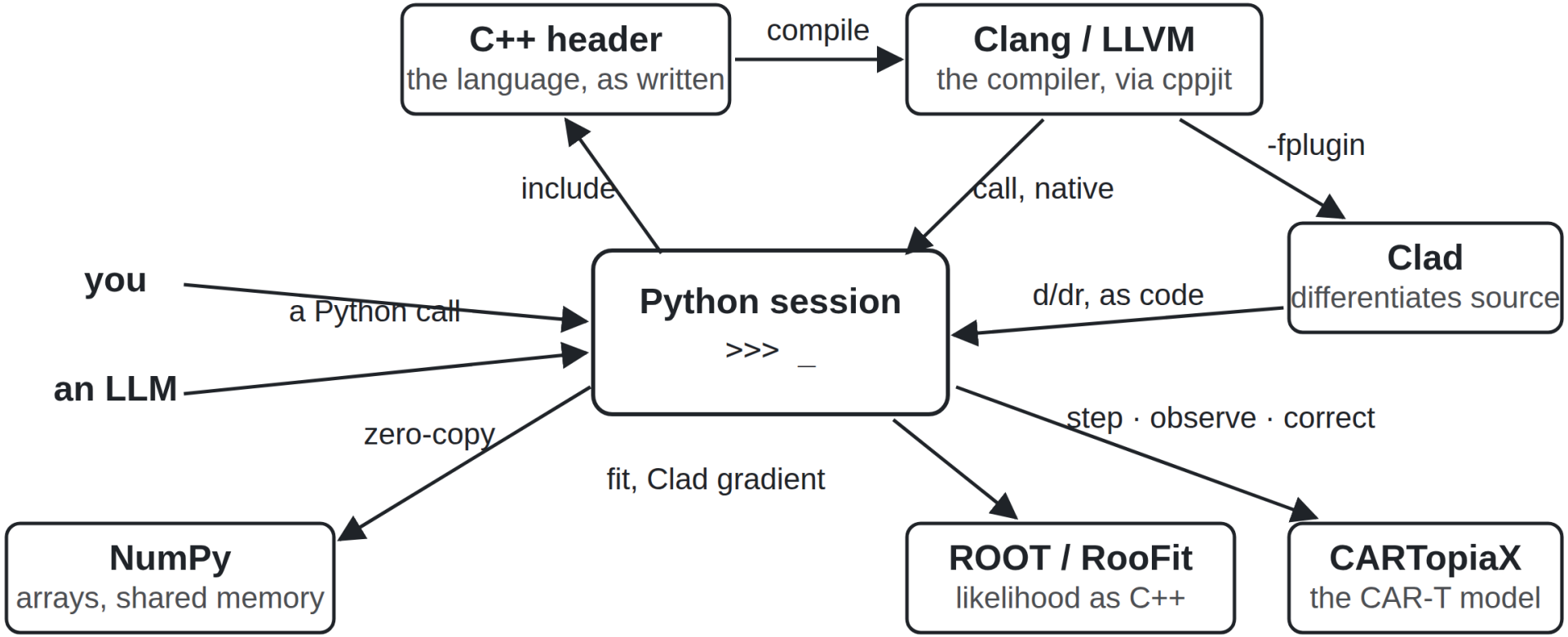
```
class SeedSecondOrganoid(sc26.PyOperation): # a BioDynaMo operation, in Python
  def Run(self):                            # the C++ scheduler calls it every step
    sim = bdm.Simulation.GetActive()
    if self.done or sim.GetScheduler().GetSimulatedSteps() < self.at_step:
      return
    ctxt = sim.GetExecutionContext()
    for pos in bdm.CreateSphereOfTumorCells(self.radius): # the model's packing
      p = bdm.Real3(pos); p[0] += self.centre[0]
      cell = bdm.TumorCell(p)                   # the model's own agent class
      cell.AddBehavior(bdm.StateControlGrowProliferate())
      ctxt.AddAgent(cell)
    self.done = True
```

```
op = bdm.Operation("seed second organoid (python)", 1)
op.AddOperationImpl(bdm.kCpu, SeedSecondOrganoid(at_step, centre, r))
sim.GetScheduler().ScheduleOp(op)# like any C++ operation
```

Steered from Python: a class inheriting BioDynaMo's operation base, scheduled like any C++ operation; at minute 30 the scheduler called it and it seeded a second organoid of 104 cells from the model's own TumorCell and behaviour classes at [160.0, 0.0, 0.0] µm. After 2 h: 208 tumour cells in the control, 208 in the treated (299 of 300 CAR T-cells alive).



# Architecture: one session, the compiler inside it




**Example 1 on the map:** the likelihood called as written (ROOT / RooFit), transformed by the compiler (Clad, a plugin of the session's Clang), composed with JAX. Example 2 walks the same three edges with a cancer model.

# Thank you!

